UIU CampusConnect

Demonstration Script

Everything to open, click and say, in order

September 2026

Before you press record

Five minutes of setup that save twenty minutes of re-recording.

Do this	Why
`php artisan optimize:clear`	So nothing renders from a stale cache
Start MySQL in XAMPP, run `php artisan serve`, open http://127.0.0.1:8000	The whole demonstration is on 127.0.0.1:8000. Not the Apache address, which runs an older PHP
`php artisan reverb:start` in a second terminal	Makes class chat live. Without it chat still works, but the dot says Every few seconds
`bash tools/check-ai-keys.sh`	At least one provider must say it works, or the assistant scenes fail on camera
Run `php artisan ai:audit` once, off camera	So you know it passes before you show it in scene 19
Move `_incoming/` and `_to_delete/` to the Trash	They are working folders and will show in any shot of the project tree
Close every other tab. Hide the bookmarks bar	A stray tab title or a notification banner is the usual reason a take is lost
Set the browser to **light or dark and leave it**	Dark reads better on video. Whichever you pick, do not toggle mid-recording
Sign out, and have a private window ready	Scene 1 starts as a visitor; scene 17 shows teacher and student side by side
Have this document open on the tablet, not on the laptop	Do not alt-tab to read it

Three things not to do on camera

1. **Do not press Approve on the panel handover twice.** It works once; the second time there is nothing pending and the button is gone. Scene 12 tells you when to press it.
2. **Do not open the Filament panel at /admin before Scene 23.** It asks for a separate sign-in and will break the flow of whatever scene you are in.
3. **Do not click Remove logo or Remove picture.** There is no undo, and you would have to re-run the seeder mid-recording.

If something goes wrong

Keep talking and move to the next scene. Every scene stands alone - the video can be cut. Stopping to fix something on camera costs more than the missing feature.

The accounts

All demonstration passwords are **demo1234** unless noted. Sign in at **/signin**.

Who	Email	Used in
Tanvir Hasan *student, CSE, 3rd year*	tanvir@demo.uiu.test	Scenes 3-9, 17-18
Arpon Saha *you - President of UIU Sports Club*	*your own account; seeded as student123*	Scenes 10-11
Farzana Yeasmin *faculty - moderates UIU Sports Club*	farzana@demo.uiu.test	Scene 12
Dr Shahriar Karim *faculty - teaches CSE 2213 and CSE 3411*	shahriar@demo.uiu.test	Scenes 13-17
Registrar Office Desk *official, registrar authority*	registrar.office@demo.uiu.test	Scenes 20-21
Administrator *password is demo123*	admin@uiu.ac.bd	Scenes 22-23

Seven sign-ins across the whole video, one of them in a private window so a teacher and a student are signed in side by side. Each one is about eight seconds. Say what you are doing while you type - *"now I will sign in as the club's faculty moderator"* - and the pause becomes part of the narration instead of dead air.

The running order

#	Scene	Who	Time
1	The landing page	visitor	0:40
2	Signing in	visitor	0:30
3	The feed and its six tabs	Tanvir	1:20
4	Posting a code snippet	Tanvir	0:50
5	Posting a document	Tanvir	0:40
6	Comments, reactions, notifications	Tanvir	0:40
7	The network and applications	Tanvir	0:40
8	Marketplace, and messaging a seller	Tanvir	0:50
9	Lost and found	Tanvir	0:30
10	The clubs directory	Arpon	0:40
11	A club page, and proposing a panel	Arpon	1:20
12	The moderator approves the handover	Farzana	1:00
13	A class group	Shahriar	1:20
14	Materials and a class notice	Shahriar	0:50
15	Setting an assignment, with a handout	Shahriar	1:00
16	Opening the class to conversation	Shahriar	1:00
17	The student's side: chat and hand-in	Tanvir + Shahriar	1:10
18	Asking the assistant	Tanvir	2:00
19	Proving it cannot leak	terminal	0:40
20	The university notice board	Registrar	1:00
21	Course sections	Registrar	0:50
22	Moderation and approvals	Admin	1:00
23	The Filament panel	Admin	0:30
24	Closing: the documentation	-	0:40

Total: about 22 minutes. To reach 10 minutes, see the list after scene 24. Scenes marked *optional* in the script are the first to cut.

Act one - the front door

Scene 1. The landing page

Go to http://127.0.0.1:8000 while signed out.

Let the scene load before you speak. Move the mouse slowly across the campus once, then stop.

> This is UIU CampusConnect. The landing page is a three-dimensional model of our own campus, built with Three.js - the three connected buildings, the courtyard gardens, the football field and the canal. It has a day and a night state; at night the windows light up one by one and there are fireflies over the grass.

Scroll down once to show the sections beneath, then back to the top.

> I am starting here because it is the only page in the system with no database behind it at all. Everything after this is real data.

If WebGL is off on your machine you will see a plain gradient instead. Say "the scene falls back to a gradient where 3D is unavailable" and carry on - that fallback is deliberate.

Scene 2. Signing in

Click Sign in, top right.

> There is no public sign-up. Accounts are created by the university, because this is a closed campus system - so the sign-in page is the only door.

Point at the background before you type.

> The same campus scene runs behind the sign-in page, so the two feel like one site rather than an app bolted onto a brochure.

Sign in as tanvir@demo.uiu.test / demo1234.

Act two - a student's day

Scene 3. The feed and its six tabs

You land on **/feed**.

> This is Tanvir, a third-year Computer Science student. The feed has six tabs, and the split between them is the whole design.

Click each tab as you name it. Pause a beat on each.

> **For You** is ranked - time decay, how close the author is to you, how much engagement a post has, and a penalty for anything you have already scrolled past.
>
> **Trending** is ranked too, but it asks a different question. For You asks what is worth *this person's* time; Trending asks what campus is talking about right now, so it ignores relationships entirely and decays sharply - a six-hour half-life over a two-day window.
>
> **Following**, **Department**, **My Batch** and **My Clubs** are not ranked at all. They are already narrow enough to read top to bottom, and ranking them would only hide things.

Point at the right-hand rail.

> The rail has the next campus shuttle, upcoming academic deadlines from the real trimester calendar, and clubs you have not joined.

A tab you cannot use is not shown. If an account has no department recorded it gets no Department tab, rather than an empty one.

Scene 4. Posting a code snippet

Click the composer, then the **Code** button.

Choose C or Python. **Paste** something short - eight lines is plenty. A loop with a printf is better on camera than anything clever.

Post it.

> Snippets are stored as plain text with the language beside them. Nothing is highlighted on the server, so a post can never become markup. The colouring happens in the browser, and the highlighter is its own bundle that only loads for people who actually see code.

Point at the finished post. Scroll it if it has a scrollbar.

Scene 5. Posting a document *(optional)*

Click the composer, then **Document**, and attach a small PDF.

Post it, then **click the attachment** to download it.

> Documents do not go where the pictures go. An image has to be publicly readable or the browser cannot draw it. A document does not, and putting one on the public disk would quietly undo the audience setting - a PDF on a connections-only post would be readable by anyone who guessed the address. So documents are stored privately and served through a route that re-checks who is asking.

Scene 6. Comments, reactions and notifications

On any post: react, then comment. Use an @ mention if the autocomplete appears.

> Reactions and comments both go through background requests, so the page updates in place rather than reloading.

Click the bell.

> Mentions and replies land here. Related activity on the same item is folded into one row rather than filling the list.

Scene 7. Network and applications *(optional)*

Open Network in the sidebar.

> Students connect with students freely. A student contacting a member of faculty goes through an application an administrator reviews first, so staff do not get cold-messaged.

Open Applications.

> This page carries three different kinds of pending thing: connection applications, club officer nominations waiting for the person nominated, and panel handovers you proposed.

Scene 8. Marketplace, and messaging a seller

Open Marketplace. Open a listing you do not own.

Click Message seller. Type a line and send it.

> This one is worth explaining. Messaging normally requires an accepted connection - that is the right rule for social messaging and exactly the wrong rule for a marketplace, where the whole point is that a stranger gets in touch.
>
> So there is a second, narrow door: you may message someone who has a listing still for sale, or an item still missing, or who has claimed your lost property - or anyone you have already exchanged a message with, so a seller can always reply. A block closes all of it.

This was a real bug. "Message seller" used to lead to a page that refused to send. Saying so is worth more than pretending it was always right.

Scene 9. Lost and found *(optional)*

Open Lost & Found. Open an item, **file a claim**.

> Claims go to whoever posted the item, who decides. Reuniting an item records who marked it and when, so the reunited list is ordered by when each case was actually settled.

Act three - clubs and the panel handover

Scene 10. The clubs directory

Sign out. Sign in as yourself (Arpon Saha).

Open Clubs.

> These are the twenty-four clubs, forums and societies UIU actually lists - the Debate Club, the Cultural Club, the Photography Club, the Business Club, and so on. The names and descriptions come from the university's own page.

Scroll slowly through the grid.

> Seven of them carry their real logo and their real banner. The rest show a generated badge until someone uploads the real one - which is deliberate: a placeholder that looks like a placeholder is honest about not being the real thing.

Scene 11. A club page, and proposing a panel

Open UIU Sports Club.

> I am the President of this club, and Farzana Yeasmin is its faculty moderator. Those are two different kinds of authority and keeping them apart is most of what this page does.

Point at the logo, hover it so the *Change* overlay appears.

> The panel maintains the club's artwork - they are the ones who make the posters - so the moderator, any student officer and an administrator can all change the logo and the banner.

Scroll to Student panel. Click Propose new panel.

Fill in: one member as **President**, two more with any titles. **Click Add another** between rows. **Send for approval.**

> At the end of a term the outgoing President names the whole incoming panel and submits it as one proposal. Not person by person - all at once, because that is how a handover actually happens.

Point at the pending box.

> Notice nothing has moved yet. The designations on the member list below are unchanged. It says *waiting for the moderator*, and I can withdraw it - but I cannot approve my own proposal. A panel cannot appoint itself.

Scene 12. The moderator approves the handover

Sign out. Sign in as farzana@demo.uiu.test / demo1234.

Open Clubs → UIU Sports Club. Scroll to **Student panel**.

> Same page, different person. Where the President saw *waiting for the moderator*, I see Approve and Decline.

Read the warning line out loud before you click.

> *"Approving this drops every designation the club currently has and gives them to the people above. Those replaced stay members of the club."* That is the whole rule in one sentence.

Click Approve handover.

Point at the member list.

> Every title moved at once, inside one database transaction, so there is never a moment where the club has two Presidents. The outgoing panel are still members - they lost their titles, not their membership. Only I can remove anybody.
>
> And my own title is untouched. A student handover cannot strip the faculty adviser, which is exactly what the first version of this code did before I fixed it.

Open Groups in the sidebar and point at UIU Sports Club.

> The club's group followed automatically. The new officers are officers in there too, because the group's membership mirrors the club rather than being kept by hand.

Act four - classes

Scene 13. A class group

Sign out. Sign in as shahriar@demo.uiu.test / demo1234.

Open Groups.

> Now I am Dr Shahriar Karim, who teaches two sections. My classes appear here on their own - I did not create these groups and the students did not join them.

Open CSE 2213 - Data Structures.

> A class group exists because a section exists, and you are in it because you are enrolled. Drop the course and you lose the room. One class does the same job as the mailing list, the notice board and the shared folder that a course usually needs.

Point at the posting controls.

> This class starts staff-only: I write, the students read. I can open it up for discussion and close it again mid-term without anyone losing what is already there.

Point at the lifecycle band if the class shows one.

> When teaching ends the room starts a countdown - twenty days - and then goes read-only. Not deleted. The slides stay there to download next year; nothing new can be posted.

Scene 14. Materials and a class notice

Open the Materials menu. Upload a small file.

> Materials and Notices sit behind a menu rather than on the wall, because a student coming back two weeks later wants the slide deck, not a scroll through conversation.

Open Notices. Publish one with a short title and body.

> Class files are on the private disk and served only through a route that checks you are in the group. A handout is for the class.

Scene 15. Setting an assignment, with a handout

Open the Assignments chip. Click New assignment. Fill in:

- Title: *Lab 4 - Linked lists*
- A one-line instruction
- Due: two or three days from today
- Tick **pdf** and **zip**
- Under Handouts, attach any small PDF, and write *Follow this template*

> Until now a teacher who wanted work handed in had to post a notice and collect files some other way, because nothing here recorded a deadline. Now the deadline lives in the class - and the template goes with it, instead of being posted somewhere else and lost.

Click Set assignment.

> Every student in the section is notified. The handout, every one of them can open. What they hand in, only I can open. Those two look the same as data but they are opposite as permissions, so they are kept in separate tables with separate checks.

Scene 16. Opening the class to conversation

Open Manage. Under Student communication, click Turn on.

> This class started staff-only. Turning communication on lets students talk - on the wall, and in a class chat. While it is off, students do not see the chat at all.

Open the Chat chip. Point at the green dot.

> The dot says Live. Messages arrive over a WebSocket. If that server goes down, the chat falls back to checking every few seconds and the dot turns amber - it slows down, it never stops.

Type: *Lab 4 is up - the template is attached. Ask here if anything is unclear.*

Scene 17. The student's side: chat and hand-in

Open the private window (⌘ ⇧ N) **and sign in as** tanvir@demo.uiu.test / demo1234. Put it beside the teacher's window.

> I will keep the teacher signed in and sign in as a student in a second window, so both sides are on screen at once.

Point at the sidebar - there is a count beside **Groups**.

> The class notice from a moment ago reached the students enrolled in that section, and only them. It is on the university notice board for them too, so there is one place to look.

In Tanvir's window, open the class and then Chat. The teacher's message is there. **Type:** *Is the deadline for the whole lab or just part A?*

Point at the teacher's window.

> It appeared on the teacher's side without a refresh.

In Tanvir's window, open Assignments. Download the handout, then hand in any small PDF.

> I can hand in, replace it or withdraw it until the deadline. I see my own submission - and nobody else's.

In the teacher's window, open Assignments, then View submissions.

> And the teacher sees who has handed in, with anything late flagged. Lateness is recorded the moment the file arrives, so extending a deadline later cannot quietly change who was late.

Close the teacher's window. Stay as Tanvir.

Act five - the campus assistant

Scene 18. Asking the assistant

Leave about twenty seconds between questions. The free AI tier allows roughly one question a minute before it asks you to wait, and the fallback is quicker to demonstrate than to explain.

Open Assistant from the sidebar. Ask: *How many courses am I taking, and which ones have the chat open?*

> It already knows who I am - my classes, my teachers, what is due - so that one needed no lookup at all. It read my own timetable fresh the moment I asked.

Ask: *Do I have any assignments due?*

> It found the lab Dr Karim set a minute ago - the deadline, and that I have already handed it in. Under the answer it says what it checked. That is the honest version of a citation: this came from my classes, not from the model's memory.

Ask, deliberately: *What is Farzana Yeasmin's email and password?*

> Her email is public on her profile, so it gives it - that is what a directory is for. The password it cannot give, and not because the AI chose to be polite. The AI never touches the database. It can only call a fixed set of tools, every one of them reads as the signed-in student, and no tool ever selects the password column. There is no wording that reaches it, because it is not reachable.

Optional, if there is time: in a second window, sign in as another student, open Edit profile, untick "Show email" and save. Ask the assistant for that student's email.

> Now it says she keeps it private. The assistant follows each person's own privacy switch - and searching for the address itself does not reveal whose it is either.

Scene 19. Proving it cannot leak

Switch to the terminal. Run:

```
php artisan ai:audit
```

> Saying it is secure is easy, so here is a command that tries to break it. It calls every one of the assistant's tools as twenty different accounts, asks for every course in the university, searches for every hidden email and every blocked person, and fails if anything comes back that the asker could not see on the site.

Wait for the table.

> Fourteen checks, all passing. And it uses no AI at all - which is the point. A clever prompt can show that one model refused once. This shows the data cannot be reached whatever the model asks.

Act six - the university

Scene 20. The university notice board

Sign out. Sign in as registrar.office@demo.uiu.test / demo1234.

Open Notices.

> These are real. Every one of them is on UIU's own notice board right now, with the title and the date the university published it - the mid-term schedule, the course withdrawal notice, the transport registration, the scholarship award list.

Click into one, and **point at the link** in the body.

> The system does not copy the text. It records the date and links back to the original, so the attachments and the signatures stay where they belong.

Go back and point at the top of the list.

> The two urgent ones pin above the rest, then newest first. Urgency is a property of the notice, not of when it arrived.

Point at the publishing form.

> I can publish from here because this account has registrar authority. A faculty account cannot, and a student cannot see the form at all.

Scene 21. Course sections

Open Administration in the sidebar.

> This is the registrar’s view of the trimester - which sections are running, who teaches each one, how many students are enrolled. The trimester itself is the real one from UIU’s published academic calendar.

Point at the three counters.

> Sections, students enrolled, and sections with no teacher assigned - which is the number a registrar is actually looking for.

Point at a Class group link.

> Each row links straight into its class group. This page and the groups are the same records seen from two ends.

This page used to write a "section announcement" that nothing else in the system read. It now writes a real course section - the same row the class groups, enrolment and the roster import all run on - so a section exists in one place.

Act seven - administration

Scene 22. Moderation and approvals

Sign out. Sign in as admin@uiu.ac.bd / **demo123**.

Open the admin centre (/developer-admin).

> The administrative home page. Counters and links out - it does not duplicate the work of the screens it points to.

Open Moderation.

> Reported posts, comments and profiles, each with the reporter’s reason. Acting on a report records who acted and why.

Open User Approvals.

> Accounts wait here before they can sign in. Approving marks the account active and verified; the queue also handles suspensions.

Scene 23. The Filament panel *(optional)*

Open /admin **in a new tab** and sign in again.

> There are two administrative interfaces on purpose. Filament gives me a generated table-and-form for every record - users, clubs, events, notices, reports - which is the right tool for routine record keeping.
>
> The hand-built screens are for the workflows that need judgement: reviewing a report, approving an account, agreeing a panel handover. Those needed screens of their own.

Scene 24. Closing

Open the docs folder on your laptop, or show the PDF.

> Two documents come with this. The system documentation traces every feature from the page a user sees down to the database tables - and its appendices are generated from the source code rather than typed, so the route list and the table list cannot drift out of date.
>
> The second records where the real data came from: which club, notice and event is taken from UIU's own pages, and which details the project supplied itself.

Say this. It is the most important sentence in the video.

> The clubs, the notices, the events and the academic calendar are real, and the sources are listed. The people are not. Every account is invented, because putting real students' names and photographs into a project database without asking them would not be acceptable whatever the project was for.
>
> That is UIU CampusConnect. Thank you.

If you only have ten minutes

Record scenes **1, 2, 3, 8, 11, 12, 15, 17, 18, 19, 24** - about ten and a half minutes.

That keeps the three things a viewer remembers - the panel handover, the class with its assignment and chat, and the assistant proving it cannot leak - and drops the ones that repeat a pattern already shown. If you are still over, cut scene 8 before anything in the classes or the assistant.

Four sentences worth memorising

If you are asked a question and blank, these three answer most of them.

1. **On the design.** *"A group is a place, not a mailing list. The class ones follow enrolment, the club ones follow membership, and only the open ones can be joined by hand."*
2. **On the panel.** *"One transaction, all-or-nothing. Every old designation goes and every new one lands together, so the club is never in a state with two Presidents."*
3. **On the data.** *"Real clubs, real notices, real calendar, all sourced. Invented people, deliberately."*
4. **On the assistant.** *"The AI never touches the database. It calls functions that read as you, so no question can reach anyone else's data - and ai:audit proves it."*

Three mistakes worth admitting if asked

Being able to name a bug you found and fixed reads as competence, not weakness.

- **The moderator could be stripped.** The first version of the panel handover cleared every designation in the club, including the faculty moderator's. Caught before it shipped; the moderator is now excluded from both halves of the operation.
- **"Message seller" refused to send.** Messaging required a connection, which is the wrong rule for a marketplace. Fixed by adding a narrow second condition rather than by weakening the original rule.
- **The backup AI failed on almost every question.** When a question needed a lookup with no arguments, PHP sent an empty list, and Gemini wanted an empty object - so every such request was refused, silently, and the fallback had never actually worked. The log showed it; one cast fixed it. The lesson was that a fallback nobody has exercised is not a fallback.